

Data Model & API Reference

NestJS 11 · PostgreSQL 16 · Prisma 6 · JWT — Harshit Makraria · 25 September 2026

Resource	Link
Live API base URL	https://events-api-43jh.onrender.com/api/v1
Swagger (interactive, try requests in the browser)	https://events-api-43jh.onrender.com/docs
OpenAPI JSON (import into Postman / Insomnia)	https://events-api-43jh.onrender.com/docs-json
Postman collection	github.com/Harshit-Makraria/rescue-rituals → docs/events-api.postman_collection.json
Source code	github.com/Harshit-Makraria/rescue-rituals
Web app using this API	rescue-rituals.vercel.app

Demo accounts (password [Password123!](#)): demo@events.dev hosts the demo events; guest@events.dev is an attendee. The API runs on a free tier: if it has been idle, the first request can take up to 30 seconds.

1. Data model	2
1.1 Relationships.....	2
1.2 Tables.....	2
1.3 Constraints and indexes.....	3
1.4 How an RSVP stays correct under load.....	4
2. API reference	5
2.1 Authentication	5
2.2 Endpoints.....	5
2.3 Examples.....	6
2.4 Errors	7
2.5 Using Swagger or Postman	7

1. Data model

Four tables in PostgreSQL. The database itself enforces the rules that matter: no duplicate RSVPs, no overbooking, valid times and secure links. Primary keys are UUIDs (safe to expose, cannot be enumerated). All timestamps are `timestamptz`, stored in UTC.

1.1 Relationships

```
users —< events      a user hosts many events      (events.creator_id → users.id)
users —< rsvps        a user makes many RSVPs      (rsvps.user_id → users.id)
events —< rsvps       an event has many RSVPs      (rsvps.event_id → events.id)
users —< notifications a user receives many notifications
events —< notifications a notification may concern an event
All foreign keys: ON DELETE CASCADE. Events are normally soft-deleted (deleted_at).
```

1.2 Tables

users

Column	Type	Rules / notes
id	uuid	Primary key, gen_random_uuid()
email	text	UNIQUE; stored lower-cased
password_hash	text	argon2id hash (never returned by the API)
name	text	Display name
refresh_token_hash	text, nullable	SHA-256 of the current refresh token; null = logged out / revoked
created_at, updated_at	timestamptz	Defaults to now()

events

Column	Type	Rules / notes
id	uuid	Primary key
creator_id	uuid	FK → users.id — the host; only they can edit or cancel
title	text	3–120 characters
description, location	text, nullable	Up to 5,000 / 200 characters
starts_at, ends_at	timestamptz	CHECK ends_at > starts_at; start must be in the future on create
capacity	int, nullable	NULL = unlimited; CHECK capacity > 0
going_count	int	Seats taken (people incl. plus-ones). CHECK 0 ≤ going_count ≤ capacity
status	enum event_status	draft published cancelled
category	enum event_category	tech music food sports arts networking outdoors other
meeting_url	text, nullable	CHECK starts with https:// — shown only to the host and people going
reminder_minutes	int, nullable	Remind attendees this long before start; CHECK 5–10,080

Column	Type	Rules / notes
reminder_sent_at	timestampz, nullable	Set when the reminder job claims the event; reset if the time changes
version	int	Optimistic-lock counter, +1 on every edit
deleted_at	timestampz, nullable	Soft delete (cancelled events keep their history)
created_at, updated_at	timestampz	

rsvps

Column	Type	Rules / notes
id	uuid	Primary key
event_id, user_id	uuid	FKs; UNIQUE (event_id, user_id) — one RSVP per person per event
status	enum rsvp_status	going waitlisted cancelled
plus_ones	int	Extra guests, CHECK 0–5; each takes a seat
phone	text, nullable	Host-only; CHECK length ≤ 30
note	text, nullable	Host-only note from the attendee; CHECK length ≤ 500
created_at	timestampz	
updated_at	timestampz	When the seat or waitlist spot was taken — defines waitlist order

notifications

Column	Type	Rules / notes
id	uuid	Primary key
user_id	uuid	FK → users.id (recipient)
event_id	uuid, nullable	FK → events.id
type	enum notification_type	event_reminder waitlist_promoted event_updated event_cancelled
title, body	text	Rendered message
read_at	timestampz, nullable	NULL = unread
created_at	timestampz	

1.3 Constraints and indexes

Constraint / index	Purpose
UNIQUE rsvps (event_id, user_id)	Duplicate RSVPs impossible, even under double taps and retries
CHECK events (going_count ≤ capacity)	The database refuses to overbook, even if application code had a bug
Partial index events (starts_at, id) WHERE published AND not deleted	The busiest query (upcoming events) is a small range scan in the same order as cursor pagination

Constraint / index	Purpose
Partial index events (category, starts_at, id)	Category browsing uses the same order
Index rsvps (event_id, status, updated_at)	Attendee lists and oldest-first waitlist promotion
Index rsvps (user_id, status)	"My RSVPs"
Partial index events (starts_at) WHERE a reminder is still owed	The reminder job scans only rows it may send
Partial UNIQUE notifications (user_id, event_id, type) WHERE type = event_reminder	At most one reminder per person per event
Index notifications (user_id, created_at DESC)	Notification inbox

Why `going\_count` is stored, not counted: event lists are read far more often than RSVPs are written, so the count is O(1) to read. It changes only inside the same transaction as the RSVP row, and the CHECK constraint keeps it honest. Schema changes ship as versioned SQL migrations ([apps/api/prisma/migrations](#)), applied automatically on deploy.

1.4 How an RSVP stays correct under load

```
-- one transaction
INSERT INTO rsvps (event_id, user_id, status) VALUES ($1, $2, 'waitlisted')
ON CONFLICT (event_id, user_id) DO UPDATE SET event_id = EXCLUDED.event_id; -- idempotent

UPDATE events SET going_count = going_count + $seats -- seats = 1 + plus_ones
WHERE id = $1 AND status = 'published' AND starts_at > now()
AND (capacity IS NULL OR going_count + $seats <= capacity);
-- 1 row updated -> going      0 rows -> waitlisted
```

Postgres row-locks the event for the UPDATE, so simultaneous requests queue and each re-checks the latest committed count: no read-then-write race and no application lock. When seats free up (cancellation, fewer plus-ones, higher capacity), the waitlist is promoted in the same transaction using `FOR UPDATE SKIP LOCKED`. Verified by tests (50 simultaneous RSVPs for 10 seats → exactly 10 going) and against the live deployment.

2. API reference

Base URL: <https://events-api-43jh.onrender.com/api/v1>. JSON in and out. The complete, always-current reference with request/response schemas is the live Swagger page (<https://events-api-43jh.onrender.com/docs>), generated from the code; a Postman collection with every request is in the repository.

2.1 Authentication

- **POST** `/auth/login` (or `/auth/register`) returns `accessToken` (valid 15 minutes) and `refreshToken` (7 days).
- Send `Authorization: Bearer <accessToken>` on protected routes.
- **POST** `/auth/refresh` exchanges a refresh token for a new pair. Refresh tokens are single-use: reusing an old one revokes the session.
- Access levels below: **Public** (no token), **JWT** (any signed-in user), **Host** (signed in and the event's creator).

2.2 Endpoints

Auth

Method	Path	Access	Description
POST	<code>/auth/register</code>	Public	Create an account → tokens + user
POST	<code>/auth/login</code>	Public	Log in → tokens + user
POST	<code>/auth/refresh</code>	Public	Rotate tokens (old refresh token stops working)
POST	<code>/auth/logout</code>	JWT	Revoke the refresh token (204)

Events

Method	Path	Access	Description
GET	<code>/events</code>	Public	Upcoming published events. Query: <code>q</code> , <code>from</code> , <code>to</code> , <code>category</code> , <code>creatorId</code> , <code>cursor</code> , <code>limit</code> (≤ 50)
GET	<code>/events/:id</code>	Public*	One event. *With a token: your RSVP status and details; meeting link if you are the host or going
POST	<code>/events</code>	JWT	Create an event; you become its host (201)
PATCH	<code>/events/:id</code>	Host	Update. Send <code>version</code> for optimistic locking (409 if stale). Attendees are notified of time/location changes
DELETE	<code>/events/:id</code>	Host	Cancel (soft delete, 204). Attendees are notified
GET	<code>/events/:id/calendar.ics</code>	Public	iCalendar file for Google / Apple / Outlook

RSVPs

Method	Path	Access	Description
POST	<code>/events/:id/rsvp</code>	JWT	Join or update your RSVP. Optional body: <code>plusOnes</code> (0–5), <code>phone</code> , <code>note</code> . Returns going or waitlisted. Idempotent
DELETE	<code>/events/:id/rsvp</code>	JWT	Leave; your seats go to the waitlist
GET	<code>/events/:id/attendees</code>	Public	People going (names and party size only), paginated

Method	Path	Access	Description
GET	/events/:id/guests	Host	Full guest list with email, phone, party size, note, status
GET	/events/:id/guests.csv	Host	Same list as a CSV download
GET	/events/:id/waitlist	Host	Waitlist in promotion order

Current user & system

Method	Path	Access	Description
GET	/users/me	JWT	Profile with hosting / going counts
GET	/users/me/events	JWT	Events you host (drafts and past included)
GET	/users/me/rsvps	JWT	Events you are going to or waitlisted for
GET	/users/me/notifications	JWT	Latest notifications + unread count (?unread=true)
POST	/users/me/notifications/read-all	JWT	Mark all as read (204)
POST	/users/me/notifications/:id/read	JWT	Mark one as read (204)
GET	/health	Public	Liveness + database check

2.3 Examples

Log in

```
POST /api/v1/auth/login
{ "email": "guest@events.dev", "password": "Password123!" }

200 OK
{ "accessToken": "eyJ...", "refreshToken": "eyJ...", "expiresIn": 900,
  "user": { "id": "0c9e...", "email": "guest@events.dev", "name": "Guest User" } }
```

Create an event

```
POST /api/v1/events Authorization: Bearer <token>
{ "title": "Postgres Performance Night", "description": "EXPLAIN ANALYZE live",
  "location": "Koramangala, Bengaluru", "startsAt": "2026-10-10T13:30:00.000Z",
  "endsAt": "2026-10-10T16:30:00.000Z", "capacity": 40, "category": "tech",
  "reminderMinutes": 60 }

201 Created
{ "id": "5b1e...", "title": "Postgres Performance Night", "capacity": 40, "goingCount": 0,
  "seatsLeft": 40, "status": "published", "version": 1, "category": "tech",
  "reminderMinutes": 60, "hasMeetingLink": false, "host": { "id": "...", "name": "Demo Host" }, ... }
```

RSVP with a guest

```
POST /api/v1/events/5b1e.../rsvp Authorization: Bearer <token>
{ "plusOnes": 1, "phone": "+91 98765 43210", "note": "Vegetarian" }

200 OK
{ "eventId": "5b1e...", "status": "going", "plusOnes": 1, "goingCount": 2, "seatsLeft": 38 }

-- when the event is full:
{ "eventId": "5b1e...", "status": "waitlisted", "plusOnes": 1, "goingCount": 40, "seatsLeft": 0 }
```

List events (cursor pagination)

```
GET /api/v1/events?category=tech&limit=20
```

```
200 OK
{ "items": [ { "id": "...", "title": "...", "startsAt": "...", "goingCount": 7, "seatsLeft": 1,
  "attendeePreview": ["Aarav Sharma", "Diya Sharma"], ... } ],
  "nextCursor": "MjAyNi0xMC0wOFQxMjozMDowMC4wMDBafDk2Y2Y..." } -- pass as ?cursor= for the next page
```

2.4 Errors

Every error has the same shape, and the `requestId` also appears in the `X-Request-Id` response header and the server log:

```
{ "statusCode": 403, "error": "FORBIDDEN", "message": "Only the host can change this event.",
  "path": "/api/v1/events/5b1e...", "timestamp": "2026-09-25T10:00:00.000Z", "requestId": "7f9c2b1e-..." }
```

Status	When
400	Validation failed (message lists each field), unknown fields sent, malformed id
401	Missing, invalid or expired access token; wrong email or password
403	Signed in but not allowed (e.g. not the host)
404	Event not found (drafts are 404 to everyone but the host)
409	Conflict: stale version, event already started or cancelled, capacity below people going, not enough seats for extra guests, email already registered
429	Rate limited (per user when signed in, per client IP otherwise)

2.5 Using Swagger or Postman

- **Swagger:** open <https://events-api-43jh.onrender.com/docs> → POST /auth/login → Try it out with a demo account → copy accessToken → Authorize → call any endpoint.
- **Postman:** import [docs/events-api.postman_collection.json](#) from the repository (base URL preset to the live API) → run Auth › login → paste the token into the collection variable `bearerToken`.